Proving SQL Queries Equivalent

U-expressions, Q-expressions and LIA*

The question

Do these two queries return the same answer on every database?

```
select x, sum(y)
from R join S on R.x = S.a
group by x
```

```
select t.x, t.r from
 (select x, sum(y) as r
  from R group by x) t
 join S on t.x = S.a
```

The question

Do these two queries return the same answer on every database?

```
select x, sum(y)
from R join S on R.x = S.a
group by x
```

```
select t.x, t.r from
 (select x, sum(y) as r
  from R group by x) t
 join S on t.x = S.a
```

Why anyone cares

Every engine rewrites your query before running it. A wrong rewrite rule returns wrong answers, silently, for years.

The question

Do these two queries return the same answer on every database?

```
select x, sum(y)
from R join S on R.x = S.a
group by x
```

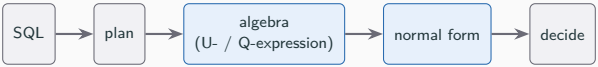
```
select t.x, t.r from
 (select x, sum(y) as r
  from R group by x) t
 join S on t.x = S.a
```

Why anyone cares

Every engine rewrites your query before running it. A wrong rewrite rule returns wrong answers, silently, for years.

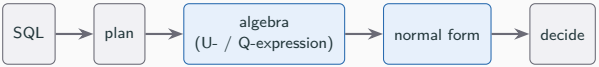
Undecidable in general $\Rightarrow$ every prover here answers only **EQUIVALENT** or **UNKNOWN**.

Three systems, one pipeline



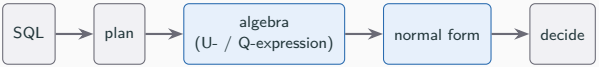
	algebra	decision step	innovation

Three systems, one pipeline



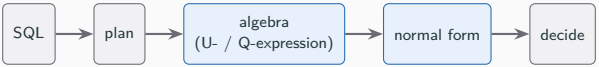
	algebra	decision step	innovation
UDP '18	U-expression	syntactic match	the foundation

Three systems, one pipeline



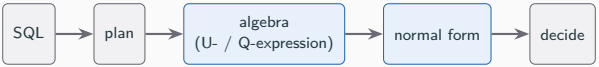
	algebra	decision step	innovation
UDP '18	U-expression	syntactic match	the foundation
QED '24	Q-expression	SMT everywhere	better modelling

Three systems, one pipeline



	algebra	decision step	innovation
UDP '18	U-expression	syntactic match	the foundation
Qed '24	Q-expression	SMT everywhere	better modelling
SQLSolver '24	U-expression	LIA*→LIA→SMT	better arithmetic

Three systems, one pipeline



	algebra	decision step	innovation
UDP '18	U-expression	syntactic match	the foundation
Qed '24	Q-expression	SMT everywhere	better modelling
SQLSolver '24	U-expression	LIA*→LIA→SMT	better arithmetic

They differ in exactly two places. Everything else follows.

Part 0

Bags, counting functions, and Σ

A table is a counting function

R

x	y
1	10
1	20
2	30

A table is a counting function

R

x	y
1	10
1	20
2	30

$R(t) =$ "how many copies of row t are in R?"

t	$R(t)$
$(1, 10)$	1
$(1, 20)$	1
$(2, 30)$	1
$(1, 99)$	0
everything else	0

A table is a counting function

R

x	y
1	10
1	20
2	30

$R(t)$ = "how many copies of row t are in R?"

t	$R(t)$
$(1, 10)$	1
$(1, 20)$	1
$(2, 30)$	1
$(1, 99)$	0
everything else	0

The whole idea

A query is a **formula that computes the count of one output row**.

Two queries are equivalent iff their formulas agree for every row, on every input.

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair
`UNION ALL`	$R(t) + S(t)$	copies accumulate

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair
`UNION ALL`	$R(t) + S(t)$	copies accumulate
`SELECT x`	$\sum_y R(x, y)$	forget $y \Rightarrow$ merge counts

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair
`UNION ALL`	$R(t) + S(t)$	copies accumulate
`SELECT x`	$\sum_y R(x, y)$	forget $y \Rightarrow$ merge counts
`DISTINCT`	$\|R(t)\|$	clamp to 0 or 1

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[\,b\,] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair
`UNION ALL`	$R(t) + S(t)$	copies accumulate
`SELECT x`	$\sum_y R(x, y)$	forget $y \Rightarrow$ merge counts
`DISTINCT`	$\|R(t)\|$	clamp to 0 or 1
`NOT EXISTS Q`	$\mathrm{not}(\sum_t Q(t))$	1 iff the count is 0

SQL becomes arithmetic

SQL	on counts	why
`FROM R`	$R(t)$	definition
`WHERE b`	$[b] \times R(t)$	multiply by 0 = delete the row
`R, S`	$R(t_1) \times S(t_2)$	every pair
`UNION ALL`	$R(t) + S(t)$	copies accumulate
`SELECT x`	$\sum_y R(x, y)$	forget $y \Rightarrow$ merge counts
`DISTINCT`	$\|R(t)\|$	clamp to 0 or 1
`NOT EXISTS Q`	$\mathrm{not}(\sum_t Q(t))$	1 iff the count is 0

No new machinery

Filtering is multiplication. Joining is multiplication. Union is addition. That is the entire trick.

Σ — “try every value and add up”

$$\sum_{y} R(1, y)$$

Σ — “try every value and add up”

$$\sum_{y} R(1, y)$$

y	$R(1, y)$
⋮	⋮
9	0
10	**1**
11	0
⋯	0
20	**1**
21	0
⋮	⋮

Σ — “try every value and add up”

$$\sum_{y} R(1, y)$$

y	$R(1, y)$
⋮	⋮
9	0
10	**1**
11	0
⋯	0
20	**1**
21	0
⋮	⋮

“Isn’t that infinite?”

No — a table has finitely many rows, so almost every term is 0.

Σ — “try every value and add up”

$$\sum_{y} R(1, y)$$

y	$R(1, y)$
⋮	⋮
9	0
10	**1**
11	0
⋯	0
20	**1**
21	0
⋮	⋮

“Isn’t that infinite?”

No — a table has finitely many rows, so almost every term is 0.

Measured in PostgreSQL:

range 0..30 → **2**

range −1000..1000 → **2**

Σ — “try every value and add up”

$$\sum_{y} R(1, y)$$

y	$R(1, y)$
⋮	⋮
9	0
10	**1**
11	0
⋯	0
20	**1**
21	0
⋮	⋮

“Isn’t that infinite?”

No — a table has finitely many rows, so almost every term is 0.

Measured in PostgreSQL:

range 0..30 → **2**

range −1000..1000 → **2**

Σ = projection

Dropping y **merges** rows that differed only in y, so their counts **add**. $\sum_y R(x, y)$ is `GROUP BY x` + `COUNT(*)`.

Two things called “sum”

	adds up	**SQL**	**on** $\{10, 20\}$
$\sum_y f(y)$	multiplicities	`COUNT(*)`	$1 + 1 = \mathbf{2}$

Two things called “sum”

	adds up	**SQL**	**on** $\{10, 20\}$
$\sum_y f(y)$	multiplicities	`COUNT(*)`	$1 + 1 = \mathbf{2}$
$\mathrm{Sum}(\lambda y.\, f(y))$	values $\times$ mult.	`SUM(y)`	$10{\cdot}1 + 20{\cdot}1 = \mathbf{30}$

Two things called “sum”

	adds up	**SQL**	**on** $\{10, 20\}$
$\sum_y f(y)$	multiplicities	`COUNT(*)`	$1 + 1 = \mathbf{2}$
$\mathrm{Sum}(\lambda y.\, f(y))$	values $\times$ mult.	`SUM(y)`	$10{\cdot}1 + 20{\cdot}1 = \mathbf{30}$

Σ is COUNT. Sum is SUM.

And $\lambda y.\, R(1, y)$ *is* a bag — the bag $\{10, 20\}$, written as its counting function. Aggregates eat whole bags, so their argument must be a function.

Two things called “sum”

	adds up	SQL	on $\{10, 20\}$
$\sum_y f(y)$	multiplicities	`COUNT(*)`	$1 + 1 = \mathbf{2}$
$\mathrm{Sum}(\lambda y.\, f(y))$	values $\times$ mult.	`SUM(y)`	$10{\cdot}1 + 20{\cdot}1 = \mathbf{30}$

Σ is COUNT. Sum is SUM.

And $\lambda y.\, R(1, y)$ *is* a bag — the bag $\{10, 20\}$, written as its counting function. Aggregates eat whole bags, so their argument must be a function.

Double every multiplicity: $\{10, 10, 20, 20\} \Rightarrow \mathrm{Sum} = \mathbf{60}$. Remember $30 \to 60$.

The running example

one rewrite, used throughout

Aggregate pushdown: Calcite's `AggregateJoinTransposeRule`

Q_1 — aggregate ABOVE the join

```
select x, sum(y) as r
from R join S on R.x = S.a
group by x
```

Aggregate pushdown: Calcite's `AggregateJoinTransposeRule`

Q_1 — aggregate ABOVE the join

```
select x, sum(y) as r
from R join S on R.x = S.a
group by x
```

```
LogicalAggregate(group=[{0}],SUMY=[SUM($1)])
  LogicalProject(X=[$0], Y=[$1])
    LogicalJoin(condition=[=($0,$2)])
      TableScan([[R]])
      TableScan([[S]])
```

Q_2 — aggregate BELOW the join

```
select t.x, t.r from
 (select x, sum(y) as r
  from R group by x) t
 join S on t.x = S.a
```

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(condition=[=($0,$2)])
    LogicalAggregate(group=[{0}],SUMY=[SUM($1)])
      TableScan([[R]])
    TableScan([[S]])
```

Aggregate pushdown: Calcite's `AggregateJoinTransposeRule`

Q_1 — aggregate ABOVE the join

```
select x, sum(y) as r
from R join S on R.x = S.a
group by x
```

```
LogicalAggregate(group=[{0}],SUMY=[SUM($1)])
  LogicalProject(X=[$0], Y=[$1])
    LogicalJoin(condition=[=($0,$2)])
      TableScan([[R]])
      TableScan([[S]])
```

Q_2 — aggregate BELOW the join

```
select t.x, t.r from
 (select x, sum(y) as r
  from R group by x) t
 join S on t.x = S.a
```

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(condition=[=($0,$2)])
    LogicalAggregate(group=[{0}],SUMY=[SUM($1)])
      TableScan([[R]])
    TableScan([[S]])
```

`Aggregate` and `Join` swap places. That is the entire rewrite.

Calcite fires the rule itself — and tells us the secret

`S.A` declared unique

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalProject(A=[$0])
      TableScan([[S]])
```

exactly Q_2

Calcite fires the rule itself — and tells us the secret

`S.A` declared unique

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalProject(A=[$0])
      TableScan([[S]])
```

exactly Q_2

`S.A` *not* unique

```
LogicalProject(X=[$0],
   $f4=[CAST(*($1, $3))])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalAggregate(agg#0=[COUNT()])
      TableScan([[S]])
```

the aggregate is multiplied by COUNT

Calcite fires the rule itself — and tells us the secret

`S.A` declared unique

```
LogicalProject(X=[$0], SUMY=[$1])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalProject(A=[$0])
      TableScan([[S]])
```

exactly Q_2

`S.A` *not* unique

```
LogicalProject(X=[$0],
   $f4=[CAST(*($1, $3))])
  LogicalJoin(cond=[=($0,$2)])
    LogicalAggregate(SUMY=[SUM($1)])
      TableScan([[R]])
    LogicalAggregate(agg#0=[COUNT()])
      TableScan([[S]])
```

the aggregate is multiplied by COUNT

A production optimiser just derived our algebra

That compensating $\times\,\mathrm{count}$ is literally the $\times\, S(a)$ factor of the U-expression. The primary key is exactly what lets a prover delete it.

One counterexample kills it

S without a key

a	*b*
1	100
1	100

One counterexample kills it

S without a key

a	*b*
1	100
1	100

	result	rows
Q_1	$(1, 60)$	1
Q_2	$(1, 30), (1, 30)$	2

One counterexample kills it

S without a key

a	b
1	100
1	100

	result	rows
Q_1	$(1, 60)$	1
Q_2	$(1, 30), (1, 30)$	2

Wrong in two independent ways:

One counterexample kills it

S without a key

a	*b*
1	100
1	100

	result	rows
Q_1	$(1, 60)$	1
Q_2	$(1, 30), (1, 30)$	2

Wrong in two independent ways:

- Q_1 joins first $\Rightarrow$ the bag entering `SUM` becomes $\{10, 10, 20, 20\}$ $\Rightarrow$ the value is corrupted $(30 \rightarrow 60)$.

One counterexample kills it

S without a key

a	*b*
1	100
1	100

	result	rows
Q_1	$(1, 60)$	1
Q_2	$(1, 30), (1, 30)$	2

Wrong in two independent ways:

- Q_1 joins first $\Rightarrow$ the bag entering `SUM` becomes $\{10, 10, 20, 20\}$ $\Rightarrow$ the value is corrupted ($30 \rightarrow 60$).
- Q_2 aggregates first $\Rightarrow$ `SUM` sees a clean bag; duplication hits the row count instead.

One counterexample kills it

S without a key

a	*b*
1	100
1	100

	result	rows
Q_1	$(1, 60)$	1
Q_2	$(1, 30), (1, 30)$	2

Wrong in two independent ways:

- Q_1 joins first $\Rightarrow$ the bag entering `SUM` becomes $\{10, 10, 20, 20\}$ $\Rightarrow$ the value is corrupted ($30 \to 60$).
- Q_2 aggregates first $\Rightarrow$ `SUM` sees a clean bag; duplication hits the row count instead.

The specification

$Q_1 \equiv Q_2$ **iff** the join column on the right is distinct. A prover that cannot turn "primary key" into algebra cannot prove this; one that proves it without the key is *unsound*.

Part I

U-expressions — UDP, VLDB 2018

The U-semiring: $(\mathcal{U}, 0, 1, +, \times, \|\cdot\|, \mathrm{not}(\cdot), \sum)$

$(\mathcal{U}, 0, 1, +, \times)$ is an ordinary commutative semiring — arithmetic without subtraction. Three operators are added.

squash $\|\cdot\|$ – models `DISTINCT`

$$\|0\| = 0 \qquad \|1 + x\| = 1$$
$$\|\|x\| + y\| = \|x + y\| \qquad \|x\| \, \|x\| = \|x\|$$
$$\|x\| \, \|y\| = \|xy\| \qquad x \, \|x\| = x$$
$$x^2 = x \;\Rightarrow\; \|x\| = x$$

The U-semiring: $(\mathcal{U}, 0, 1, +, \times, \|\cdot\|, \mathrm{not}(\cdot), \sum)$

$(\mathcal{U}, 0, 1, +, \times)$ is an ordinary commutative semiring — arithmetic without subtraction. Three operators are added.

squash $\|\cdot\|$ – models `DISTINCT`

$$\|0\| = 0 \qquad \|1 + x\| = 1$$
$$\|\|x\| + y\| = \|x + y\| \qquad \|x\|\,\|x\| = \|x\|$$
$$\|x\|\,\|y\| = \|xy\| \qquad x\,\|x\| = x$$
$$x^2 = x \;\Rightarrow\; \|x\| = x$$

negation $\mathrm{not}(\cdot)$ – models `NOT EXISTS`

$$\mathrm{not}(0) = 1 \qquad \mathrm{not}(x+y) = \mathrm{not}(x)\mathrm{not}(y)$$
$$\mathrm{not}(xy) = \|\mathrm{not}(x)+\mathrm{not}(y)\| \qquad \mathrm{not}(\|x\|) = \mathrm{not}(x)$$

The U-semiring: $(\mathcal{U}, 0, 1, +, \times, \|\cdot\|, \mathrm{not}(\cdot), \sum)$

$(\mathcal{U}, 0, 1, +, \times)$ is an ordinary commutative semiring — arithmetic without subtraction. Three operators are added.

squash $\|\cdot\|$ – models `DISTINCT`

$$\|0\| = 0 \qquad \|1 + x\| = 1$$
$$\|\|x\| + y\| = \|x + y\| \qquad \|x\|\,\|x\| = \|x\|$$
$$\|x\|\,\|y\| = \|xy\| \qquad x\,\|x\| = x$$
$$x^2 = x \;\Rightarrow\; \|x\| = x$$

negation $\mathrm{not}(\cdot)$ – models `NOT EXISTS`

$$\mathrm{not}(0) = 1 \qquad \mathrm{not}(x+y) = \mathrm{not}(x)\mathrm{not}(y)$$
$$\mathrm{not}(xy) = \|\mathrm{not}(x)+\mathrm{not}(y)\| \qquad \mathrm{not}(\|x\|) = \mathrm{not}(x)$$

Why so few, and why all equations?

Fewer axioms = smaller trusted base = simpler proof search. And every axiom is an **identity** $E_1 = E_2$, so it can be used as a rewrite rule and checked in Coq. An order relation $\leq$ was deliberately excluded.

. . . and unbounded summation

summation $\sum$ – models projection

$$\sum_t (f_1 + f_2) = \sum_t f_1 + \sum_t f_2$$

$$\sum_{t_1} \sum_{t_2} f = \sum_{t_2} \sum_{t_1} f$$

$$x \sum_t f = \sum_t xf$$

$$\left\| \sum_t f \right\| = \left\| \sum_t \|f\| \right\|$$

. . . and unbounded summation

summation $\sum$ – models projection

$$\sum_t (f_1 + f_2) = \sum_t f_1 + \sum_t f_2$$

$$\sum_{t_1} \sum_{t_2} f = \sum_{t_2} \sum_{t_1} f$$

$$x \sum_t f = \sum_t xf$$

$$\left\| \sum_t f \right\| = \left\| \sum_t \|f\| \right\|$$

equality

$$[\, e_1 = e_2 \,] + [\, e_1 \neq e_2 \,] = 1$$

$$f(e_1)[\, e_1 = e_2 \,] = f(e_2)[\, e_1 = e_2 \,]$$

$$\sum_t [\, t = e \,] = 1$$

. . . and unbounded summation

summation $\sum$ – models projection

$$\sum_t (f_1 + f_2) = \sum_t f_1 + \sum_t f_2$$

$$\sum_{t_1} \sum_{t_2} f = \sum_{t_2} \sum_{t_1} f$$

$$x \sum_t f = \sum_t xf$$

$$\left\| \sum_t f \right\| = \left\| \sum_t \|f\| \right\|$$

equality

$$[\, e_1 = e_2 \,] + [\, e_1 \neq e_2 \,] = 1$$

$$f(e_1)[\, e_1 = e_2 \,] = f(e_2)[\, e_1 = e_2 \,]$$

$$\sum_t [\, t = e \,] = 1$$

The workhorse (derived from the three equality axioms)

$$\sum_t [\, t = e \,] \times f(t) = f(e)$$

A summation whose variable is pinned collapses to one value.

. . . and unbounded summation

summation $\sum$ – models projection

$$\sum_t (f_1 + f_2) = \sum_t f_1 + \sum_t f_2$$

$$\sum_{t_1} \sum_{t_2} f = \sum_{t_2} \sum_{t_1} f$$

$$x \sum_t f = \sum_t xf$$

$$\left\| \sum_t f \right\| = \left\| \sum_t \|f\| \right\|$$

equality

$$[\, e_1 = e_2 \,] + [\, e_1 \neq e_2 \,] = 1$$

$$f(e_1)[\, e_1 = e_2 \,] = f(e_2)[\, e_1 = e_2 \,]$$

$$\sum_t [\, t = e \,] = 1$$

The workhorse (derived from the three equality axioms)

$$\sum_t [\, t = e \,] \times f(t) = f(e)$$

A summation whose variable is pinned collapses to one value.

A join condition `ON x = a` creates exactly such a pin — which is why join conditions make summations disappear.

Keys as an *equation*, not an implication

$$[\, t.k = t'.k \,] \times R(t) \times R(t') \;=\; [\, t = t' \,] \times R(t)$$

Keys as an *equation*, not an implication

$$[\, t.k = t'.k \,] \times R(t) \times R(t') \;=\; [\, t = t' \,] \times R(t)$$

- Set $t = t'$: $\quad R(t)^2 = R(t) \quad \Rightarrow \quad R(t) \in \{0, 1\}$ — no duplicate rows

Keys as an *equation*, not an implication

$$[\,t.k = t'.k\,] \times R(t) \times R(t') \;=\; [\,t = t'\,] \times R(t)$$

- Set $t = t'$: $\quad R(t)^2 = R(t) \quad \Rightarrow \quad R(t) \in \{0, 1\}$ — no duplicate rows
- Take $t \neq t'$ with $t.k = t'.k$: $\quad \text{RHS} = 0 \quad \Rightarrow \quad R(t)R(t') = 0$ — no two rows share a key

Keys as an *equation*, not an implication

$$[\,t.k = t'.k\,] \times R(t) \times R(t') \;=\; [\,t = t'\,] \times R(t)$$

- Set $t = t'$: $\quad R(t)^2 = R(t) \quad \Rightarrow \quad R(t) \in \{0, 1\}$ — no duplicate rows
- Take $t \neq t'$ with $t.k = t'.k$: $\quad \text{RHS} = 0 \quad \Rightarrow \quad R(t)R(t') = 0$ — no two rows share a key

Why an equation?

Everything in a U-semiring must be an **identity** $E_1 = E_2$, so it can be used as a rewrite rule and checked in Coq. The obvious definition "$\sum_t R(t)[\,t.k = e\,] \leq 1$" needs an *order relation* — extra axioms the authors refused to add.

A complete proof, by axiom-chasing

R has key k; I is an index on $R.a$; goal `SELECT * FROM R WHERE a>=12` $\equiv$ index lookup.

$$[\![Q_2]\!](t) = \sum_{t_1,t_2,t_3} [\,t_2{=}t\,][\,t_1.k{=}t_2.k\,][\,t_1.a{\geq}12\,][\,t_3.k{=}t_1.k\,][\,t_3.a{=}t_1.a\,]R(t_3)R(t_2)$$

A complete proof, by axiom-chasing

R has key k; I is an index on $R.a$; goal `SELECT * FROM R WHERE a>=12` $\equiv$ index lookup.

$$[\![Q_2]\!](t) = \sum_{t_1,t_2,t_3} [\,t_2{=}t\,][\,t_1.k{=}t_2.k\,][\,t_1.a{\geq}12\,][\,t_3.k{=}t_1.k\,][\,t_3.a{=}t_1.a\,]R(t_3)R(t_2)$$

1. t_1 has schema (k, a), so the two equalities determine t_1. Workhorse lemma deletes $\sum_{t_1}$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.k{=}t_2.k\,][\,t_3.a{\geq}12\,]R(t_3)R(t_2)$$

A complete proof, by axiom-chasing

R has key k; I is an index on $R.a$; goal `SELECT * FROM R WHERE a>=12` $\equiv$ index lookup.

$$[\![Q_2]\!](t) = \sum_{t_1,t_2,t_3} [\,t_2{=}t\,][\,t_1.k{=}t_2.k\,][\,t_1.a{\geq}12\,][\,t_3.k{=}t_1.k\,][\,t_3.a{=}t_1.a\,]R(t_3)R(t_2)$$

1. t_1 has schema (k, a), so the two equalities determine t_1. Workhorse lemma deletes $\sum_{t_1}$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.k{=}t_2.k\,][\,t_3.a{\geq}12\,]R(t_3)R(t_2)$$

2. KEY identity on $[\,t_3.k{=}t_2.k\,]R(t_3)R(t_2) = [\,t_2{=}t_3\,]R(t_2)$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.a{\geq}12\,]R(t_2)[\,t_2{=}t_3\,]$$

A complete proof, by axiom-chasing

R has key k; I is an index on $R.a$; goal `SELECT * FROM R WHERE a>=12` $\equiv$ index lookup.

$$[\![Q_2]\!](t) = \sum_{t_1,t_2,t_3} [\,t_2{=}t\,][\,t_1.k{=}t_2.k\,][\,t_1.a{\geq}12\,][\,t_3.k{=}t_1.k\,][\,t_3.a{=}t_1.a\,]R(t_3)R(t_2)$$

1. t_1 has schema (k, a), so the two equalities determine t_1. Workhorse lemma deletes $\sum_{t_1}$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.k{=}t_2.k\,][\,t_3.a{\geq}12\,]R(t_3)R(t_2)$$

2. KEY identity on $[\,t_3.k{=}t_2.k\,]R(t_3)R(t_2) = [\,t_2{=}t_3\,]R(t_2)$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.a{\geq}12\,]R(t_2)[\,t_2{=}t_3\,]$$

3. Workhorse on t_3, then t_2: $\quad [\![Q_2]\!](t) = [\,t.a \geq 12\,] \times R(t) = [\![Q_1]\!](t)$ ■

A complete proof, by axiom-chasing

R has key k; I is an index on $R.a$; goal `SELECT * FROM R WHERE a>=12` $\equiv$ index lookup.

$$[\![Q_2]\!](t) = \sum_{t_1,t_2,t_3} [\,t_2{=}t\,][\,t_1.k{=}t_2.k\,][\,t_1.a{\geq}12\,][\,t_3.k{=}t_1.k\,][\,t_3.a{=}t_1.a\,]R(t_3)R(t_2)$$

1. t_1 has schema (k, a), so the two equalities determine t_1. Workhorse lemma deletes $\sum_{t_1}$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.k{=}t_2.k\,][\,t_3.a{\geq}12\,]R(t_3)R(t_2)$$

2. KEY identity on $[\,t_3.k{=}t_2.k\,]R(t_3)R(t_2) = [\,t_2{=}t_3\,]R(t_2)$:

$$\sum_{t_2,t_3} [\,t_2{=}t\,][\,t_3.a{\geq}12\,]R(t_2)[\,t_2{=}t_3\,]$$

3. Workhorse on t_3, then t_2: $\quad [\![Q_2]\!](t) = [\,t.a \geq 12\,] \times R(t) = [\![Q_1]\!](t)$ ■

The strategy

SPNF → delete pinned summations → apply constraint identities → compare **syntactically**.

UDP's two ceilings

1. Syntactic predicates

Comparison is congruence closure over *uninterpreted* functions.

$$[\,a + b = 0\,] \quad \text{vs} \quad [\,a = -b\,]$$

Does not match. No arithmetic.

UDP's two ceilings

1. Syntactic predicates

Comparison is congruence closure over *uninterpreted* functions.

$$[\,a + b = 0\,] \quad \text{vs} \quad [\,a = -b\,]$$

Does not match. No arithmetic.

$\Rightarrow$ every system after 2018 calls an SMT solver here.

UDP's two ceilings

1. Syntactic predicates

Comparison is congruence closure over *uninterpreted* functions.

$$[\,a + b = 0\,] \quad \text{vs} \quad [\,a = -b\,]$$

Does not match. No arithmetic.

$\Rightarrow$ every system after 2018 calls an SMT solver here.

2. Aggregates compared out of context

$\mathrm{Sum}(E_1) = \mathrm{Sum}(E_2)$ only by congruence, i.e. only if $E_1 = E_2$:

$$\lambda y.\, R(x,y)\,\|S(x)\| \stackrel{?}{=} \lambda y.\, R(x,y)$$

Genuinely false standalone — if $x \notin S$ the left is empty.

UDP's two ceilings

1. Syntactic predicates

Comparison is congruence closure over *uninterpreted* functions.

$$[\,a + b = 0\,] \quad \text{vs} \quad [\,a = -b\,]$$

Does not match. No arithmetic.

$\Rightarrow$ every system after 2018 calls an SMT solver here.

2. Aggregates compared out of context

$\mathrm{Sum}(E_1) = \mathrm{Sum}(E_2)$ only by congruence, i.e. only if $E_1 = E_2$:

$$\lambda y.\, R(x,y)\,\|S(x)\| \stackrel{?}{=} \lambda y.\, R(x,y)$$

Genuinely false standalone — if $x \notin S$ the left is empty.

$\Rightarrow$ QED's ambient-assumption trick fixes exactly this.

UDP's two ceilings

1. Syntactic predicates

Comparison is congruence closure over *uninterpreted* functions.

$$[\,a + b = 0\,] \quad \text{vs} \quad [\,a = -b\,]$$

Does not match. No arithmetic.

$\Rightarrow$ every system after 2018 calls an SMT solver here.

2. Aggregates compared out of context

$\mathrm{Sum}(E_1) = \mathrm{Sum}(E_2)$ only by congruence, i.e. only if $E_1 = E_2$:

$$\lambda y.\, R(x,y)\,\|S(x)\| \stackrel{?}{=} \lambda y.\, R(x,y)$$

Genuinely false standalone — if $x \notin S$ the left is empty.

$\Rightarrow$ QED's ambient-assumption trick fixes exactly this.

Also: no NULLs (hence no outer joins); `UDP` bails when the two sides have different numbers of terms; the constraint chase may not terminate.

Score: 33–34 of 232 Calcite pairs.

Part II

Q-expressions — QED, VLDB 2024

One decision changes everything

Same syntax. One concrete model instead of an axiom system.

$$\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}, \qquad [P] = \begin{cases} 1 \\ 0 \end{cases}, \qquad \|a\| \coloneqq [\, a \neq 0 \,], \qquad \mathrm{not}(a) \coloneqq [\, a = 0 \,]$$

One decision changes everything

Same syntax. One concrete model instead of an axiom system.

$$\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}, \qquad [P] = \begin{cases} 1 \\ 0 \end{cases}, \qquad \|a\| \coloneqq [\, a \neq 0 \,], \qquad \mathrm{not}(a) \coloneqq [\, a = 0 \,]$$

U-expression

$\|\cdot\|$ is an opaque symbol with 6 axioms.

To prove anything you *search for an axiom chain*.

One decision changes everything

Same syntax. One concrete model instead of an axiom system.

$$\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}, \qquad [\,P\,] = \begin{cases} 1 \\ 0 \end{cases}, \qquad \|a\| \coloneqq [\,a \neq 0\,], \qquad \mathrm{not}(a) \coloneqq [\,a = 0\,]$$

U-expression

$\|\cdot\|$ is an opaque symbol with 6 axioms.
To prove anything you *search for an axiom chain*.

Q-expression

$\|S(a)\|$ *is* the Boolean $S(a) \neq 0$ —
an uninterpreted predicate.
To prove anything you *call Z3*.

One decision changes everything

Same syntax. One concrete model instead of an axiom system.

$$\overline{\mathbb{N}} = \mathbb{N} \cup \{\infty\}, \qquad [P] = \begin{cases} 1 \\ 0 \end{cases}, \qquad \|a\| \coloneqq [\, a \neq 0 \,], \qquad \mathrm{not}(a) \coloneqq [\, a = 0 \,]$$

U-expression

$\|\cdot\|$ is an opaque symbol with 6 axioms.
To prove anything you *search for an axiom chain*.

Q-expression

$\|S(a)\|$ *is* the Boolean $S(a) \neq 0$ —
an uninterpreted predicate.
To prove anything you *call Z3*.

Nothing is lost

$\overline{\mathbb{N}}$ *is* a U-semiring. QED simply declines the extra generality in exchange for an SMT encoding.

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\| R'(k) \right\| \times [\, s = f_R(k) \,]$$

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\| R'(k) \right\| \times [\, s = f_R(k) \,]$$

(when the key is the whole schema: $R \rightsquigarrow \lambda k.\ \|R(k)\|$)

Provably complete: the rewritten problem is equivalent, not merely implied.

Composes with join, group-by, everything — *no per-shape rules*.

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\| R'(k) \right\| \times [\, s = f_R(k) \,]$$

(when the key is the whole schema: $R \rightsquigarrow \lambda k.\ \|R(k)\|$)

Provably complete: the rewritten problem is equivalent, not merely implied.

Composes with join, group-by, everything — *no per-shape rules*.

Foreign key — one global formula

$$\forall b, k.\ \|S(b,k)\| \to \left\| R'(k) \right\|$$

UDP rewrites eagerly and loops on cyclic FKs; QED lets the solver unfold lazily.

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\|R'(k)\right\| \times [\, s = f_R(k) \,]$$

(when the key is the whole schema: $R \rightsquigarrow \lambda k.\ \|R(k)\|$)

Provably complete: the rewritten problem is equivalent, not merely implied.

Composes with join, group-by, everything — *no per-shape rules*.

Foreign key — one global formula

$$\forall b, k.\ \|S(b,k)\| \to \left\|R'(k)\right\|$$

UDP rewrites eagerly and loops on cyclic FKs; Qed lets the solver unfold lazily.

NULL — lift every type to `Option(`T`)`

Strict by default, with the documented exceptions:

$$a \texttt{ And } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{T}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{T}, b, a))$$

$$a \texttt{ Or } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{F}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{F}, b, a))$$

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\| R'(k) \right\| \times [\, s = f_R(k) \,]$$

(when the key is the whole schema: $R \rightsquigarrow \lambda k.\ \|R(k)\|$)

Provably complete: the rewritten problem is equivalent, not merely implied.

Composes with join, group-by, everything — *no per-shape rules*.

Foreign key — one global formula

$$\forall b, k.\ \|S(b,k)\| \to \left\| R'(k) \right\|$$

UDP rewrites eagerly and loops on cyclic FKs; QED lets the solver unfold lazily.

NULL — lift every type to `Option(`T`)`

Strict by default, with the documented exceptions:

$$a \texttt{ And } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{T}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{T}, b, a))$$

$$a \texttt{ Or } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{F}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{F}, b, a))$$

Check: Null `And` False = False. Strictness would get that wrong.

Modelling upgrades

Primary key — rewrite the *table*

$$R \rightsquigarrow \lambda k, s.\ \left\|R'(k)\right\| \times [\, s = f_R(k) \,]$$

(when the key is the whole schema: $R \rightsquigarrow \lambda k.\ \|R(k)\|$)

Provably complete: the rewritten problem is equivalent, not merely implied.

Composes with join, group-by, everything — *no per-shape rules*.

Foreign key — one global formula

$$\forall b, k.\ \|S(b,k)\| \to \left\|R'(k)\right\|$$

UDP rewrites eagerly and loops on cyclic FKs; QED lets the solver unfold lazily.

NULL — lift every type to `Option(`T`)`

Strict by default, with the documented exceptions:

$$a \texttt{ And } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{T}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{T}, b, a))$$

$$a \texttt{ Or } b = \mathrm{ite}(a{=}\mathsf{N}, \mathrm{ite}(b{=}\mathsf{F}, \mathsf{N}, b), \mathrm{ite}(a{=}\mathsf{F}, b, a))$$

Check: Null `And` False = False. Strictness would get that wrong.

Unknown operators — QOp/HOp first class.

`LIMIT`, `IN`, anything unmodelled becomes an uninterpreted operator that still participates via congruence, instead of aborting the proof.

$$[\![\texttt{GroupBy}(k, \alpha(f), Q)]\!] = \lambda x, y. \underbrace{\left\| \sum_s [x{=}k(s)]\, Q(s) \right\|}_{\text{the group is non-empty}} \times \underbrace{\left[y = \alpha(\lambda y'. \sum_s Q(s)[x{=}k(s) \wedge y'{=}f(s)]) \right]}_{y \text{ is the aggregate of the group's bag}}$$

$$[\![\texttt{GroupBy}(k, \alpha(f), Q)]\!] = \lambda x, y. \underbrace{\left\| \sum_s [x{=}k(s)]\, Q(s) \right\|}_{\text{the group is non-empty}} \times \underbrace{\left[y = \alpha(\lambda y'. \sum_s Q(s)[x{=}k(s) \wedge y'{=}f(s)]) \right]}_{y \text{ is the aggregate of the group's bag}}$$

- The squash is why `GROUP BY` emits **one** row per key whatever the input multiplicities.

GROUP BY, written out

$$[\![\texttt{GroupBy}(k, \alpha(f), Q)]\!] = \lambda x, y. \underbrace{\left\| \sum_s [x{=}k(s)]\, Q(s) \right\|}_{\text{the group is non-empty}} \times \underbrace{\left[y = \alpha(\lambda y'.\ \sum_s Q(s)[x{=}k(s) \wedge y'{=}f(s)]) \right]}_{y \text{ is the aggregate of the group's bag}}$$

- The squash is why `GROUP BY` emits **one** row per key whatever the input multiplicities.
- α is an HOp — uninterpreted. QED knows only: **equal bags** $\Rightarrow$ **equal aggregates**.

GROUP BY, written out

$$[\![\texttt{GroupBy}(k, \alpha(f), Q)]\!] = \lambda x, y. \underbrace{\left\| \sum_s [x{=}k(s)]\, Q(s) \right\|}_{\text{the group is non-empty}} \times \underbrace{\left[y = \alpha(\lambda y'. \sum_s Q(s)[x{=}k(s) \wedge y'{=}f(s)]) \right]}_{y \text{ is the aggregate of the group's bag}}$$

- The squash is why `GROUP BY` emits **one** row per key whatever the input multiplicities.
- α is an HOp — uninterpreted. QED knows only: **equal bags** $\Rightarrow$ **equal aggregates**.

The trap UDP fell into

UDP de-sugars `GROUP BY` into a correlated subquery over *every* row of R — no squash. A key occurring twice then yields two output rows. Multiplicities silently wrong.

Proving the running example — step 1: stabilise

Translate, with the key already applied:

$$Q_1(x, r) = \left\| \sum_{y,a} [x{=}a]\, R(x, y)\, \|S(a)\| \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ \sum_{a} [x{=}a]\, R(x, y)\, \|S(a)\|) \right]$$

$$Q_2(x, r) = \sum_{a} [x{=}a] \left\| \sum_{y} R(x, y) \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ R(x, y)) \right] \times \|S(a)\|$$

Proving the running example — step 1: stabilise

Translate, with the key already applied:

$$Q_1(x, r) = \left\| \sum_{y,a} [x{=}a]\, R(x, y)\, \|S(a)\| \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ \sum_{a} [x{=}a]\, R(x, y)\, \|S(a)\|) \right]$$

$$Q_2(x, r) = \sum_{a} [x{=}a] \left\| \sum_{y} R(x, y) \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ R(x, y)) \right] \times \|S(a)\|$$

Both contain $[x = a]$, which pins a to x. QED finds it by asking the SMT solver for congruence classes — x and a land in the same class — then deletes $\sum_a$:

Proving the running example — step 1: stabilise

Translate, with the key already applied:

$$Q_1(x,r) = \left\| \sum_{y,a} [x{=}a]\, R(x,y)\, \|S(a)\| \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ \sum_{a} [x{=}a]\, R(x,y)\, \|S(a)\|) \right]$$

$$Q_2(x,r) = \sum_{a} [x{=}a] \left\| \sum_{y} R(x,y) \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)) \right] \times \|S(a)\|$$

Both contain $[x = a]$, which pins a to x. QED finds it by asking the SMT solver for congruence classes — x and a land in the same class — then deletes $\sum_a$:

$$Q_1(x,r) = \left\| \sum_{y} R(x,y)\, \|S(x)\| \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)\, \|S(x)\|) \right] \right\|$$

$$Q_2(x,r) = \left\| \sum_{y} R(x,y)\, \|S(x)\| \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)) \right] \right\|$$

Proving the running example — step 1: stabilise

Translate, with the key already applied:

$$Q_1(x,r) = \left\| \sum_{y,a} [x{=}a]\, R(x,y)\, \|S(a)\| \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ \sum_{a} [x{=}a]\, R(x,y)\, \|S(a)\|) \right]$$

$$Q_2(x,r) = \sum_{a} [x{=}a] \left\| \sum_{y} R(x,y) \right\| \times \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)) \right] \times \|S(a)\|$$

Both contain $[x = a]$, which pins a to x. QED finds it by asking the SMT solver for congruence classes — x and a land in the same class — then deletes $\sum_a$:

$$Q_1(x,r) = \left\| \sum_{y} R(x,y)\, \|S(x)\| \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)\, \|S(x)\|) \right] \right\|$$

$$Q_2(x,r) = \left\| \sum_{y} R(x,y)\, \|S(x)\| \left[r = \mathrm{Sum}(\lambda y.\ R(x,y)) \right] \right\|$$

One difference left: the $\|S(x)\|$ *inside* the aggregate.

(UDP could only find the pin if it were syntactically visible — write the join as `x - a = 0` and it fails.)

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_1, \qquad \phi_2 = \exists y.\, R(x,y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y)\neq 0 \wedge S(x)\neq 0 \wedge r = v_1, \qquad \phi_2 = \exists y.\, R(x,y)\neq 0 \wedge S(x)\neq 0 \wedge r = v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

- Needs $\lambda y.\, R(x,y)\,\|S(x)\| \equiv \lambda y.\, R(x,y)$ — FALSE standalone ($x \notin S \Rightarrow$ left is empty). UDP and SPES stop here.

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_1, \qquad \phi_2 = \exists y.\, R(x,y) \neq 0 \wedge S(x) \neq 0 \wedge r = v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

- Needs $\lambda y.\, R(x,y)\,\|S(x)\| \equiv \lambda y.\, R(x,y)$ — FALSE standalone ($x \notin S \Rightarrow$ left is empty). UDP and SPES stop here.
- QED runs the recursive check under the ambient assumption $\phi_1 \vee \phi_2$.

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y){\neq}0 \wedge S(x){\neq}0 \wedge r{=}v_1, \qquad \phi_2 = \exists y.\, R(x,y){\neq}0 \wedge S(x){\neq}0 \wedge r{=}v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

- Needs $\lambda y.\, R(x,y)\,\|S(x)\| \equiv \lambda y.\, R(x,y)$ — FALSE standalone ($x \notin S \Rightarrow$ left is empty). UDP and SPES stop here.
- QED runs the recursive check under the ambient assumption $\phi_1 \vee \phi_2$.
- *Sound because:* if $\neg\phi_1 \wedge \neg\phi_2$, both terms are 0 and already equal — so assuming the disjunction loses nothing.

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y){\neq}0 \wedge S(x){\neq}0 \wedge r{=}v_1, \qquad \phi_2 = \exists y.\, R(x,y){\neq}0 \wedge S(x){\neq}0 \wedge r{=}v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

- Needs $\lambda y.\, R(x,y)\,\|S(x)\| \equiv \lambda y.\, R(x,y)$ — FALSE standalone ($x \notin S \Rightarrow$ left is empty). UDP and SPES stop here.
- QED runs the recursive check under the ambient assumption $\phi_1 \vee \phi_2$.
- *Sound because:* if $\neg\phi_1 \wedge \neg\phi_2$, both terms are 0 and already equal — so assuming the disjunction loses nothing.
- $\phi_1 \vee \phi_2$ contains $S(x) \neq 0$, hence $\|S(x)\| = 1$, hence $\forall y.\ R(x,y)\,\|S(x)\| = R(x,y)$. $v_1 = v_2$. Done. ■

Step 2–3: encode, then the trick

Everything is under a squash, so only *zero vs non-zero* matters. Push $\neq 0$ inward ($\sum \rightsquigarrow \exists$, $\times \rightsquigarrow \wedge$, $+ \rightsquigarrow \vee$):

$$\phi_1 = \exists y.\, R(x,y)\neq 0 \wedge S(x)\neq 0 \wedge r{=}v_1, \qquad \phi_2 = \exists y.\, R(x,y)\neq 0 \wedge S(x)\neq 0 \wedge r{=}v_2$$

Identical except v_1 vs v_2. So: is $v_1 = v_2$?

- Needs $\lambda y.\, R(x,y)\,\|S(x)\| \equiv \lambda y.\, R(x,y)$ — FALSE standalone ($x \notin S \Rightarrow$ left is empty). UDP and SPES stop here.
- QED runs the recursive check under the ambient assumption $\phi_1 \vee \phi_2$.
- *Sound because:* if $\neg\phi_1 \wedge \neg\phi_2$, both terms are 0 and already equal — so assuming the disjunction loses nothing.
- $\phi_1 \vee \phi_2$ contains $S(x) \neq 0$, hence $\|S(x)\| = 1$, hence $\forall y.\ R(x,y)\,\|S(x)\| = R(x,y)$. $v_1 = v_2$. Done. ■

In English

The group-by only emits a row for an x that survived the join, and any such x is in S — so inside the aggregate $\|S(x)\|$ is just 1 and vanishes.

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express.

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express. **Why it works — the elegant bit:**

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express. **Why it works — the elegant bit:**

- Linearised normal form fixes a total order on the summation variables of each table.

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express. **Why it works — the elegant bit:**

- Linearised normal form fixes a total order on the summation variables of each table.
- Then a group of terms with scope $R_1^{k_1} \cdots R_m^{k_m}$ evaluates to a monomial $B_S \times n_1^{k_1} \cdots n_m^{k_m}$, where n_i is a free multiplicity.

Also: a complete fragment

$\mathcal{F}_T$

`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express. **Why it works — the elegant bit:**

- Linearised normal form fixes a total order on the summation variables of each table.
- Then a group of terms with scope $R_1^{k_1} \cdots R_m^{k_m}$ evaluates to a monomial $B_S \times n_1^{k_1} \cdots n_m^{k_m}$, where n_i is a free multiplicity.
- Bag equivalence becomes polynomial identity, and two polynomials agree iff all coefficients do.

Also: a complete fragment

$\mathcal{F}_T$

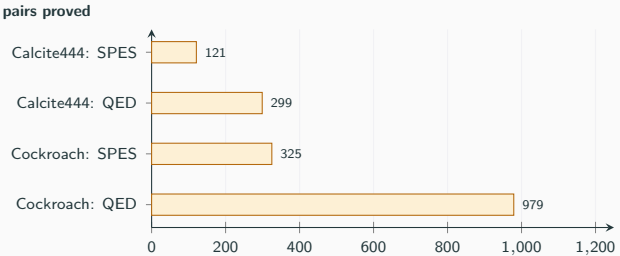
`Table`, `Values`, `Filter`, `Proj`, `Join`, `Union`, with predicates and projections in any first-order theory T, plus primary keys.

Generalises Cohen's UCQ result: UCQ forces conjunctions of (in)equalities and bare column references; $\mathcal{F}_T$ allows anything T can express. **Why it works — the elegant bit:**

- Linearised normal form fixes a total order on the summation variables of each table.
- Then a group of terms with scope $R_1^{k_1} \cdots R_m^{k_m}$ evaluates to a monomial $B_S \times n_1^{k_1} \cdots n_m^{k_m}$, where n_i is a free multiplicity.
- Bag equivalence becomes polynomial identity, and two polynomials agree iff all coefficients do.

Cost: linearisation enumerates orderings — $1, 1, 3, 13, 75, 541, \ldots$ (ordered Bell numbers). Use it only inside the fragment.

Qed results

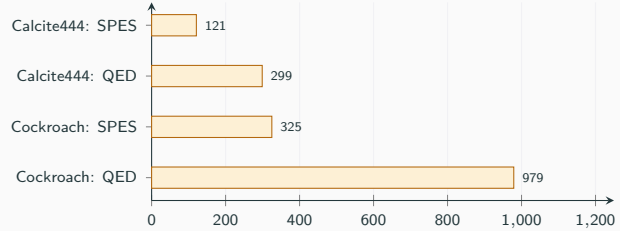


Suites: Calcite v1.32 (444 pairs), CockroachDB (1287).

Older 232-pair Calcite suite: UDP 34, SPES 95, QED 147.

Qed results

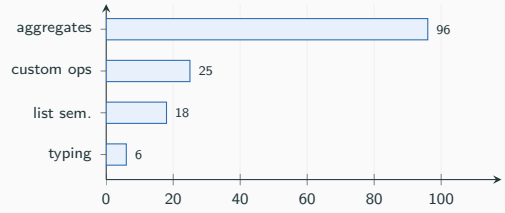
pairs proved



Suites: Calcite v1.32 (444 pairs), CockroachDB (1287).

Older 232-pair Calcite suite: UDP 34, SPES 95, Qed 147.

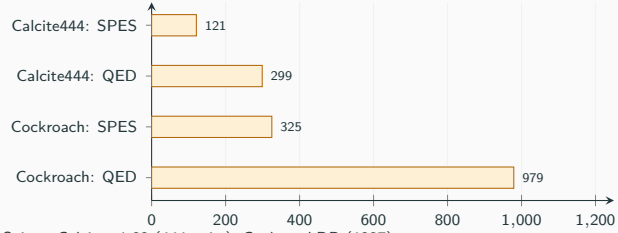
where Qed fails (Calcite)



Aggregates dominate — the direct cost of leaving α uninterpreted.

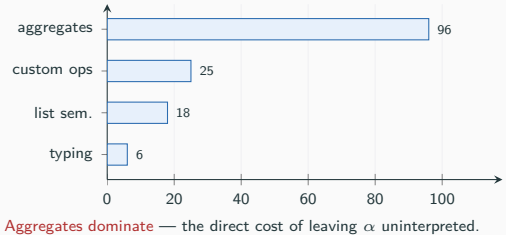
Qed results

pairs proved



Suites: Calcite v1.32 (444 pairs), CockroachDB (1287).

where Qed fails (Calcite)



Aggregates dominate — the direct cost of leaving α uninterpreted.

Older 232-pair Calcite suite: UDP 34, SPES 95, QED 147.

Every pair SPES proves, QED also proves. ∼0.88 s average, mostly SMT time.

Part III

U-expressions + LIA* — SQLSolver, SIGMOD 2024

A sharper diagnosis: the unbounded summation

Equivalence = unsatisfiability of $\exists t.\, f_1(t) \neq f_2(t)$. If f_1, f_2 were plain arithmetic, Z3 would settle it. They contain $\sum$, and no SMT solver has a theory of unbounded sums.

A sharper diagnosis: the unbounded summation

Equivalence = unsatisfiability of $\exists t.\, f_1(t) \neq f_2(t)$. If f_1, f_2 were plain arithmetic, Z3 would settle it. They contain $\sum$, and no SMT solver has a theory of unbounded sums.

```
Q1: (select x from R where y<=1000)
    union all (select x from R where y>1000 and z<500)
Q2:  select x from R where y<=1000 or z<500
```

A sharper diagnosis: the unbounded summation

Equivalence = unsatisfiability of $\exists t.\, f_1(t) \neq f_2(t)$. If f_1, f_2 were plain arithmetic, Z3 would settle it. They contain $\sum$, and no SMT solver has a theory of unbounded sums.

```
Q1: (select x from R where y<=1000)
    union all (select x from R where y>1000 and z<500)
Q2:  select x from R where y<=1000 or z<500
```

$$f_1 = \sum_{t_1} E_1 + \sum_{t_1} E_2 \qquad \text{vs} \qquad f_2 = \sum_{t_1} E_3$$

A sharper diagnosis: the unbounded summation

Equivalence = unsatisfiability of $\exists t.\, f_1(t) \neq f_2(t)$. If f_1, f_2 were plain arithmetic, Z3 would settle it. They contain $\sum$, and no SMT solver has a theory of unbounded sums.

```
Q1: (select x from R where y<=1000)
    union all (select x from R where y>1000 and z<500)
Q2:  select x from R where y<=1000 or z<500
```

$$f_1 = \sum_{t_1} E_1 + \sum_{t_1} E_2 \qquad \text{vs} \qquad f_2 = \sum_{t_1} E_3$$

Two summations on the left, one on the right

UDP: rejects — term counts differ. WeTune: no rule for $\sum + \sum$.

On Calcite, 180 pairs have unbounded sums; UDP handles 33, WeTune 71.

(Qed, published later, *does* get this one — it is inside its complete fragment. LIA*'s advantage is that it needs no fragment, no shared scope, and survives aggregates and nesting.)

LIA*: put the unboundedness where a solver can reach it

Additive closure

$$S^* = \left\{ \vec{v} \;\middle|\; \vec{v} = \sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \right\}$$

all non-negative combinations of members of S, with n arbitrary.

LIA*: put the unboundedness where a solver can reach it

Additive closure

$$S^* = \left\{ \vec{v} \;\middle|\; \vec{v} = \sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \right\}$$

all non-negative combinations of members of S, with n arbitrary.

$\sum_t f(t)$ = add up f over an unknown, unbounded number of tuples

S^* = add up an unknown, unbounded number of members of S

LIA*: put the unboundedness where a solver can reach it

Additive closure

$$S^* = \left\{ \vec{v} \;\middle|\; \vec{v} = \sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \right\}$$

all non-negative combinations of members of S, with n arbitrary.

$\sum_t f(t)$ = add up f over an unknown, unbounded number of tuples

S^* = add up an unknown, unbounded number of members of S

Let S be the set of possible per-tuple contribution vectors. Then the value of the summation is exactly an element of S^*.

LIA*: put the unboundedness where a solver can reach it

Additive closure

$$S^* = \left\{ \vec{v} \;\middle|\; \vec{v} = \textstyle\sum_{i=1}^{n} \lambda_i \vec{v}_i,\ \vec{v}_i \in S,\ \lambda_i \geq 0 \right\}$$

all non-negative combinations of members of S, with n arbitrary.

$$\textstyle\sum_t f(t) = \text{add up } f \text{ over an unknown, unbounded number of tuples}$$
$$S^* \;=\; \text{add up an unknown, unbounded number of members of } S$$

Let S be the set of possible per-tuple contribution vectors. Then the value of the summation is exactly an element of S^*.

$$\text{LIA}^* : \quad \exists \vec{u}, \vec{v}.\ F_1(\vec{u}, \vec{v}) \wedge \vec{v} \in \{\vec{x} \mid F_2(\vec{x})\}^*$$

NP-complete, and reducible to plain LIA.

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.
2. Vectorise: name each sum v_i and write them as *one* vector sum

$$\exists \vec{v}.\ v_1 + v_2 \neq v_3 \wedge (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3)$$

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.
2. Vectorise: name each sum v_i and write them as *one* vector sum

$$\exists \vec{v}.\ v_1{+}v_2 \neq v_3 \wedge (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3)$$

3. Replace the vector sum by the closure: $(v_1, v_2, v_3) \in \{(x_1, x_2, x_3) \mid x_i{=}E_i\}^*$

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.
2. Vectorise: name each sum v_i and write them as *one* vector sum

$$\exists \vec{v}.\ v_1{+}v_2 \neq v_3 \wedge (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3)$$

3. Replace the vector sum by the closure: $(v_1, v_2, v_3) \in \{(x_1, x_2, x_3) \mid x_i{=}E_i\}^*$
4. Encode U-terms into LIA: $[\,b\,] \mapsto \mathrm{ite}(b, 1, 0)$, $\|E\| \mapsto \mathrm{ite}(E{=}0, 0, 1)$, every $R(t)$/attribute $\mapsto$ a fresh integer.

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.
2. Vectorise: name each sum v_i and write them as *one* vector sum

$$\exists \vec{v}.\ v_1{+}v_2 \neq v_3 \wedge (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3)$$

3. Replace the vector sum by the closure: $(v_1, v_2, v_3) \in \{(x_1, x_2, x_3) \mid x_i{=}E_i\}^*$
4. Encode U-terms into LIA: $[\,b\,] \mapsto \mathrm{ite}(b, 1, 0)$, $\|E\| \mapsto \mathrm{ite}(E{=}0, 0, 1)$, every $R(t)$/attribute $\mapsto$ a fresh integer.
5. Compute generators and emit LIA. Here the `ite` definitions force $x_1{+}x_2 = x_3$, whose generators are $(1, 0, 1)$ and $(0, 1, 1)$:

$$\vec{v} = \lambda_1(1, 0, 1) + \lambda_2(0, 1, 1) = (\lambda_1, \lambda_2, \lambda_1{+}\lambda_2)$$

which forces $v_3 = v_1{+}v_2$ — contradicting $v_1{+}v_2 \neq v_3$. Unsat $\Rightarrow$ equivalent.

Five steps

1. Goal: $\sum_{t_1} E_1 + \sum_{t_1} E_2 \neq \sum_{t_1} E_3$ unsatisfiable.
2. Vectorise: name each sum v_i and write them as *one* vector sum

$$\exists \vec{v}.\ v_1{+}v_2 \neq v_3 \wedge (v_1, v_2, v_3) = \sum_{t_1} (E_1, E_2, E_3)$$

3. Replace the vector sum by the closure: $(v_1, v_2, v_3) \in \{(x_1, x_2, x_3) \mid x_i{=}E_i\}^*$
4. Encode U-terms into LIA: $[\,b\,] \mapsto \mathrm{ite}(b, 1, 0)$, $\|E\| \mapsto \mathrm{ite}(E{=}0, 0, 1)$, every $R(t)$/attribute $\mapsto$ a fresh integer.
5. Compute generators and emit LIA. Here the `ite` definitions force $x_1{+}x_2 = x_3$, whose generators are $(1, 0, 1)$ and $(0, 1, 1)$:

$$\vec{v} = \lambda_1(1, 0, 1) + \lambda_2(0, 1, 1) = (\lambda_1, \lambda_2, \lambda_1{+}\lambda_2)$$

which forces $v_3 = v_1{+}v_2$ — contradicting $v_1{+}v_2 \neq v_3$. Unsat $\Rightarrow$ equivalent.

What just happened

No structural matching at all. SQLSolver reasoned *arithmetically* about three sums sharing an index. Structural provers cannot even express that step.

Concrete aggregates — the other half

Every aggregate gets the same skeleton...

$$f(t_l \cdot t_r) = \underbrace{[\, p(t_r) \,]}_{\texttt{HAVING}} \times \underbrace{\left\| \sum_{t_1} R(t_1)[\, t_l{=}x(t_1) \,] \right\|}_{\text{group non-empty}} \times \underbrace{(\cdots)}_{\text{the aggregate}}$$

Concrete aggregates — the other half

Every aggregate gets the same skeleton…

$$f(t_l \cdot t_r) = \underbrace{[\,p(t_r)\,]}_{\texttt{HAVING}} \times \underbrace{\left\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,] \right\|}_{\text{group non-empty}} \times \underbrace{(\cdots)}_{\text{the aggregate}}$$

- Sum: $\left[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}\,])\,y(t_1)\right]$ — literally $\sum_y y \times$ mult.

Concrete aggregates — the other half

Every aggregate gets the same skeleton…

$$f(t_l \cdot t_r) = \underbrace{[\,p(t_r)\,]}_{\texttt{HAVING}} \times \underbrace{\left\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,] \right\|}_{\text{group non-empty}} \times \underbrace{(\cdots)}_{\text{the aggregate}}$$

- Sum: $\left[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}\,])\,y(t_1)\right]$ — literally $\sum_y y \times$ mult.
- Max: not computed but characterised — "nothing exceeds t_r" *and* "something equals t_r".

Concrete aggregates — the other half

Every aggregate gets the same skeleton…

$$f(t_l \cdot t_r) = \underbrace{[\,p(t_r)\,]}_{\texttt{HAVING}} \times \underbrace{\Big\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,] \Big\|}_{\text{group non-empty}} \times \underbrace{(\cdots)}_{\text{the aggregate}}$$

- Sum: $\big[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}\,])\,y(t_1)\big]$ — literally $\sum_y y \times$ mult.
- Max: not computed but characterised — "nothing exceeds t_r" *and* "something equals t_r".
- Avg: no division; assert $t_r \times \mathrm{count} = \mathrm{sum}$.

Concrete aggregates — the other half

Every aggregate gets the same skeleton…

$$f(t_l \cdot t_r) = \underbrace{[\,p(t_r)\,]}_{\texttt{HAVING}} \times \underbrace{\Big\| \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,] \Big\|}_{\text{group non-empty}} \times \underbrace{(\cdots)}_{\text{the aggregate}}$$

- Sum: $\big[t_r = \sum_{t_1} R(t_1)[\,t_l{=}x(t_1)\,]\mathrm{not}([\,\mathit{IsNull}\,])\,y(t_1)\big]$ — literally $\sum_y y \times$ mult.
- Max: not computed but characterised — "nothing exceeds t_r" *and* "something equals t_r".
- Avg: no division; assert $t_r \times \mathrm{count} = \mathrm{sum}$.

The payoff

`select a, count(b) from R group by a` $\equiv$ `sum` of those counts regrouped.

No uninterpreted-aggregate prover can ever do this — the needed fact is a fact *about* Sum and Count.

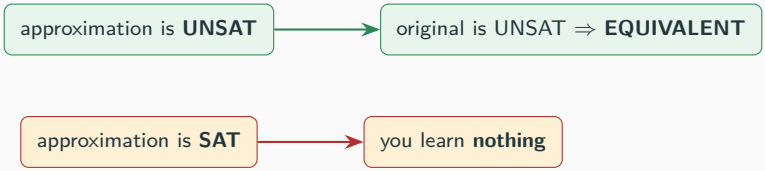
The catch: over-approximation

Joins produce $\sum(x_1 \times x_2)$ — a product of two variables, forbidden in LIA. Fix: replace it by a fresh y constrained only by $y{=}0 \leftrightarrow (x_1{=}0 \vee x_2{=}0)$.



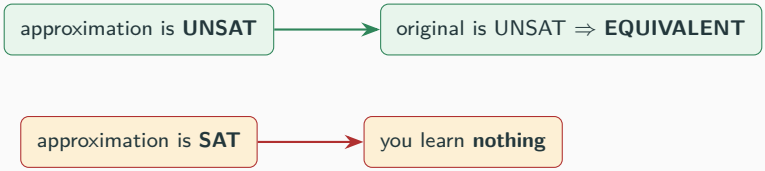
The catch: over-approximation

Joins produce $\sum(x_1 \times x_2)$ — a product of two variables, forbidden in LIA. Fix: replace it by a fresh y constrained only by $y{=}0 \leftrightarrow (x_1{=}0 \vee x_2{=}0)$.



The catch: over-approximation

Joins produce $\sum(x_1 \times x_2)$ — a product of two variables, forbidden in LIA. Fix: replace it by a fresh y constrained only by $y{=}0 \leftrightarrow (x_1{=}0 \vee x_2{=}0)$.

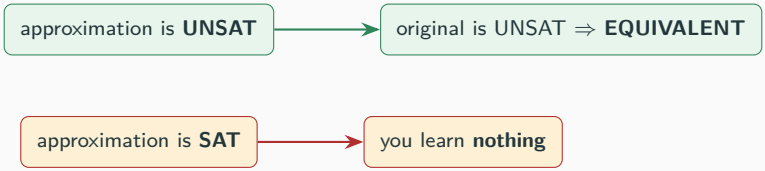


Implementer's warning

A SAT result is not a counterexample. Report "unknown". If you report "not equivalent" your tool is unsound.

The catch: over-approximation

Joins produce $\sum(x_1 \times x_2)$ — a product of two variables, forbidden in LIA. Fix: replace it by a fresh y constrained only by $y{=}0 \leftrightarrow (x_1{=}0 \vee x_2{=}0)$.

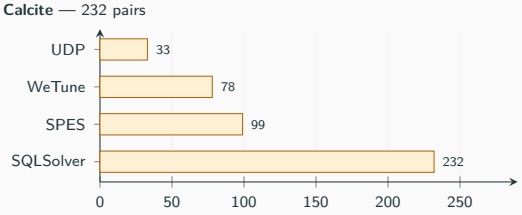


Implementer's warning

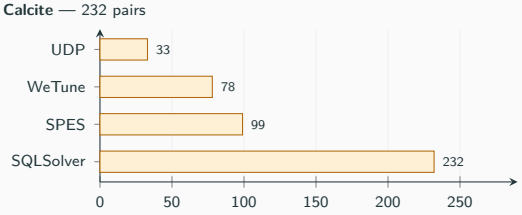
A SAT result is not a counterexample. Report "unknown". If you report "not equivalent" your tool is unsound.

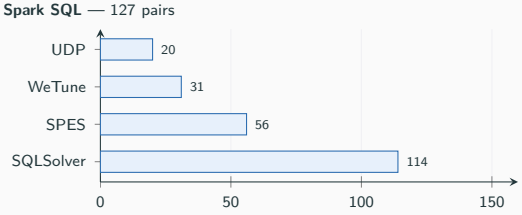
Also: `ORDER BY` is handled by a separate, *syntactic* divide-and-conquer layer — U-expressions model bags, which have no order.

SQLSolver results

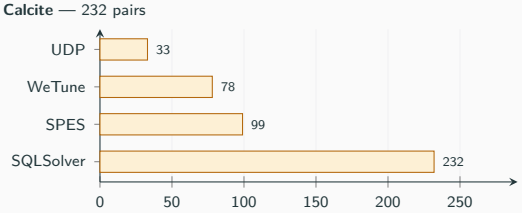


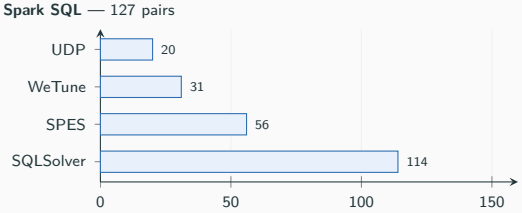
SQLSolver results





SQLSolver results



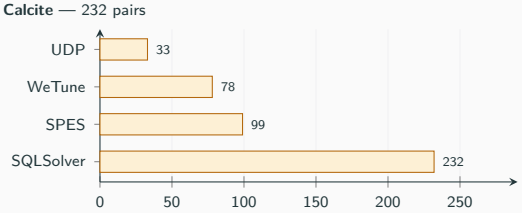


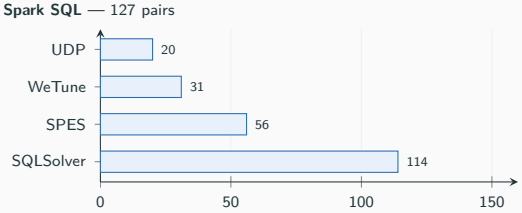
TPC-C (19): SPES 15 → SQLSolver 19

TPC-H (22): SPES 0 → SQLSolver 19

UDP and WeTune were not evaluated on TPC-C/TPC-H.

SQLSolver results





TPC-C (19): SPES 15 → SQLSolver 19 **TPC-H** (22): SPES 0 → SQLSolver 19

UDP and WeTune were not evaluated on TPC-C/TPC-H. Of 400 pairs the three prior provers prove 207 between them; SQLSolver proves 384, a *superset*. 16 proved by nobody.

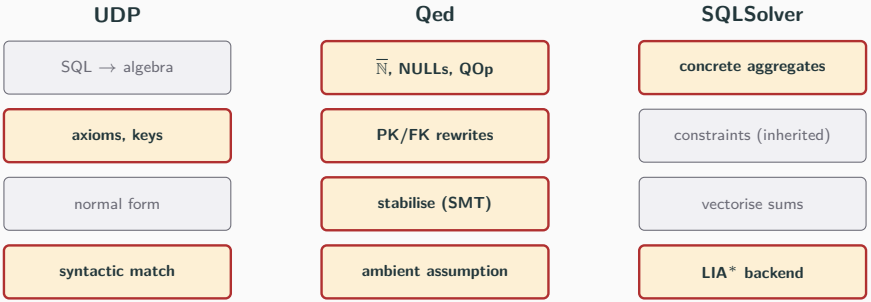
Comparison

and what nobody has built yet

Where each system spent its effort

UDP	Qed	SQLSolver
SQL $\longrightarrow$ algebra	$\overline{\mathbb{N}}$, NULLs, QOp	concrete aggregates
axioms, keys	PK/FK rewrites	constraints (inherited)
normal form	stabilise (SMT)	vectorise sums
syntactic match	ambient assumption	LIA* backend

Where each system spent its effort



Qed rebuilt the modelling and the search. SQLSolver rebuilt the aggregates and the arithmetic. They are nearly orthogonal.

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes
sum of counts = one count	no	**no**	yes

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes
sum of counts = one count	no	**no**	yes
redundant self-join on a key	no	yes	–

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes
sum of counts = one count	no	**no**	yes
redundant self-join on a key	no	yes	–
predicate pushdown, `LEFT JOIN`	no	yes	partial

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes
sum of counts = one count	no	**no**	yes
redundant self-join on a key	no	yes	–
predicate pushdown, `LEFT JOIN`	no	yes	partial
anything with `ORDER BY`	no	no	yes

A gallery of separating examples

	UDP	QED	SQLSOLVER
equal-but-different predicates	no	yes	yes
aggregate pushdown through a key join	no	yes	likely
$\sum + \sum = \sum$ (split union)	no	only in $\mathcal{F}_T$	yes
sum of counts = one count	no	**no**	yes
redundant self-join on a key	no	yes	–
predicate pushdown, `LEFT JOIN`	no	yes	partial
anything with `ORDER BY`	no	no	yes

Rows 3 and 4 are the interesting ones

Row 4 is a *modelling* gap (uninterpreted aggregates); row 3 is *structural* (QED's general algorithm needs a term bijection; only its complete-fragment algorithm relaxes that). SQLSOLVER's two contributions are exactly these two rows.

The root cause: squashed or not

Squashed

`DISTINCT`, `GROUP BY`, `EXISTS`

Multiplicities are irrelevant. Push $\neq 0$ inward:

$$\sum \rightsquigarrow \exists, \quad \times \rightsquigarrow \wedge, \quad + \rightsquigarrow \vee$$

$\Rightarrow$ a pure Boolean formula. Cheap and strong.

The root cause: squashed or not

Squashed

`DISTINCT`, `GROUP BY`, `EXISTS`

Multiplicities are irrelevant. Push $\neq 0$ inward:

$$\sum \rightsquigarrow \exists, \quad \times \rightsquigarrow \wedge, \quad + \rightsquigarrow \vee$$

$\Rightarrow$ a pure Boolean formula. Cheap and strong.

Not squashed

plain projection, join, `UNION ALL`

Counts *are* the answer, so both compare summands pointwise:

$$(\forall \vec{s}.\, f = g) \Rightarrow \sum_{\vec{s}} f = \sum_{\vec{s}} g$$

UDP Alg. 2 and QED Alg. 6 need a 1-to-1 term match; QED Alg. 3 only needs a shared relational scope, but is limited to $\mathcal{F}_T$.

The root cause: squashed or not

Squashed

`DISTINCT`, `GROUP BY`, `EXISTS`

Multiplicities are irrelevant. Push $\neq 0$ inward:

$$\sum \rightsquigarrow \exists, \quad \times \rightsquigarrow \wedge, \quad + \rightsquigarrow \vee$$

$\Rightarrow$ a pure Boolean formula. Cheap and strong.

Not squashed

plain projection, join, `UNION ALL`

Counts *are* the answer, so both compare summands pointwise:

$$(\forall \vec{s}.\, f = g) \Rightarrow \sum_{\vec{s}} f = \sum_{\vec{s}} g$$

UDP Alg. 2 and QED Alg. 6 need a 1-to-1 term match; QED Alg. 3 only needs a shared relational scope, but is limited to $\mathcal{F}_T$.

Why it matters

Under a squash, $+$ becomes $\vee$ and the whole comparison is Boolean — an SMT solver settles it immediately. Without a squash the very same rewrite is an arithmetic statement about sums, and you need either alignable scopes or LIA*.

The root cause: squashed or not

Squashed

`DISTINCT`, `GROUP BY`, `EXISTS`

Multiplicities are irrelevant. Push $\neq 0$ inward:

$$\sum \rightsquigarrow \exists, \quad \times \rightsquigarrow \wedge, \quad + \rightsquigarrow \vee$$

$\Rightarrow$ a pure Boolean formula. Cheap and strong.

Not squashed

plain projection, join, `UNION ALL`

Counts *are* the answer, so both compare summands pointwise:

$$(\forall \vec{s}.\, f = g) \Rightarrow \sum_{\vec{s}} f = \sum_{\vec{s}} g$$

UDP Alg. 2 and QED Alg. 6 need a 1-to-1 term match; QED Alg. 3 only needs a shared relational scope, but is limited to $\mathcal{F}_T$.

Why it matters

Under a squash, $+$ becomes $\vee$ and the whole comparison is Boolean — an SMT solver settles it immediately. Without a squash the very same rewrite is an arithmetic statement about sums, and you need either alignable scopes or LIA*.

SQLSolver sidesteps the whole dichotomy: LIA* reasons about the sums' *values*, so no alignment is ever needed.

Do not trust cross-paper numbers

reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Do not trust cross-paper numbers

reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Same baseline, same nominal benchmark, different answers — because the extraction pipelines differ, so the suites differ.

Do not trust cross-paper numbers

reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Same baseline, same nominal benchmark, different answers — because the extraction pipelines differ, so the suites differ.

- QED's harder suites (444 Calcite v1.32, 1287 CockroachDB) were never run through SQLSOLVER, and vice versa.

Do not trust cross-paper numbers

reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Same baseline, same nominal benchmark, different answers — because the extraction pipelines differ, so the suites differ.

- QED's harder suites (444 Calcite v1.32, 1287 CockroachDB) were never run through SQLSOLVER, and vice versa.
- QED does not evaluate against SQLSOLVER at all — near-simultaneous work.

Do not trust cross-paper numbers

reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Same baseline, same nominal benchmark, different answers — because the extraction pipelines differ, so the suites differ.

- QED's harder suites (444 Calcite v1.32, 1287 CockroachDB) were never run through SQLSOLVER, and vice versa.
- QED does not evaluate against SQLSOLVER at all — near-simultaneous work.
- "Proves 100% of Calcite" means *100% of the pairs this group extracted and chose to model*.

Do not trust cross-paper numbers

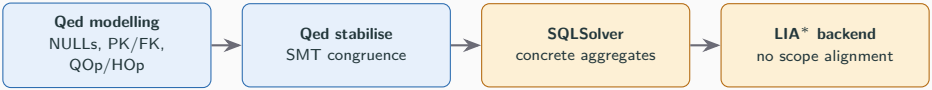
reported by	benchmark	SPES scores
QED	"232 Calcite pairs"	95
SQLSOLVER	"232 Calcite pairs"	99

Same baseline, same nominal benchmark, different answers — because the extraction pipelines differ, so the suites differ.

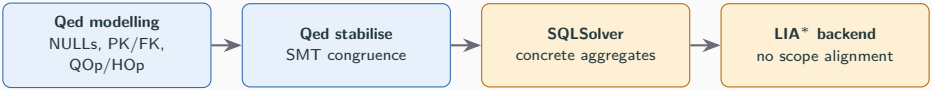
- QED's harder suites (444 Calcite v1.32, 1287 CockroachDB) were never run through SQLSOLVER, and vice versa.
- QED does not evaluate against SQLSOLVER at all — near-simultaneous work.
- "Proves 100% of Calcite" means *100% of the pairs this group extracted and chose to model*.

The only defensible comparison is one you run yourself, one extraction, all tools, same input. Nobody has done it.

The system nobody has built

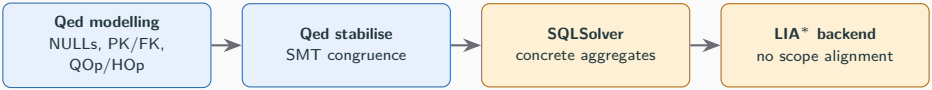


The system nobody has built



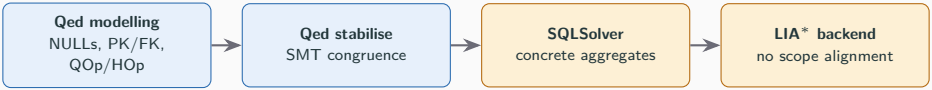
- QED's term-bijection step is its ceiling. LIA* removes exactly that ceiling.

The system nobody has built



- QED's term-bijection step is its ceiling. LIA* removes exactly that ceiling.
- QED's uninterpreted HOp is its largest failure bucket (96 Calcite cases). SQLSOLVER's aggregate tables fill it.

The system nobody has built



- QED's term-bijection step is its ceiling. LIA* removes exactly that ceiling.
- QED's uninterpreted HOp is its largest failure bucket (96 Calcite cases). SQLSOLVER's aggregate tables fill it.
- SQLSOLVER's weakest areas — NULLs, constraints, unknown operators — are precisely QED's contributions.

If you are implementing this

Build order (do not start with the fancy parts):

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation
4. bring in SMT for predicates — biggest capability jump per line of code

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation
4. bring in SMT for predicates — biggest capability jump per line of code
5. stabilisation via congruence classes — second biggest

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation
4. bring in SMT for predicates — biggest capability jump per line of code
5. stabilisation via congruence classes — second biggest
6. primary keys (one rewrite, large payoff), then squash / `GroupBy`

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation
4. bring in SMT for predicates — biggest capability jump per line of code
5. stabilisation via congruence classes — second biggest
6. primary keys (one rewrite, large payoff), then squash / `GroupBy`
7. only then: NULLs, foreign keys, unknown operators, LIA*

If you are implementing this

Build order (do not start with the fancy parts):

1. plan → algebra for `Scan/Filter/Project/Join/Union` only
2. normalisation to sum-of-products (pure syntax, no solver)
3. pointwise term matching with scope permutation
4. bring in SMT for predicates — biggest capability jump per line of code
5. stabilisation via congruence classes — second biggest
6. primary keys (one rewrite, large payoff), then squash / `GroupBy`
7. only then: NULLs, foreign keys, unknown operators, LIA*

Test against a real database

Generate tiny instances with heavy duplicates and NULLs; compare with `EXCEPT ALL` both ways; and evaluate your algebra directly on the same instance to validate the *translation*, not just the decision procedure.

Our running example must produce $(1, 60)$ vs two copies of $(1, 30)$. If yours doesn't, the `GroupBy` or key rule is wrong.

Three ideas worth remembering

Takeaways

1. **A table is a counting function.** Filtering is multiplication, union is addition, projection is an unbounded sum. Every system here is a different way of comparing two such formulas.

Takeaways

1. **A table is a counting function.** Filtering is multiplication, union is addition, projection is an unbounded sum. Every system here is a different way of comparing two such formulas.

2. **The two 2024 systems solve different halves.**
   QED = better *model* of SQL (NULLs, keys, unknown operators, context-sensitive subquery checks).
   SQLSOLVER = better *arithmetic* (unbounded sums via LIA*, real aggregates).

Takeaways

1. **A table is a counting function.** Filtering is multiplication, union is addition, projection is an unbounded sum. Every system here is a different way of comparing two such formulas.

2. **The two 2024 systems solve different halves.**
   QED = better *model* of SQL (NULLs, keys, unknown operators, context-sensitive subquery checks).
   SQLSOLVER = better *arithmetic* (unbounded sums via LIA*, real aggregates).

3. **Squash is the dividing line.** Under a squash, equivalence is a Boolean question an SMT solver answers instantly. Without one, it is an arithmetic question about sums — and that is where the frontier is.

Takeaways

1. **A table is a counting function.** Filtering is multiplication, union is addition, projection is an unbounded sum. Every system here is a different way of comparing two such formulas.

2. **The two 2024 systems solve different halves.**
   QED = better *model* of SQL (NULLs, keys, unknown operators, context-sensitive subquery checks).
   SQLSOLVER = better *arithmetic* (unbounded sums via LIA*, real aggregates).

3. **Squash is the dividing line.** Under a squash, equivalence is a Boolean question an SMT solver answers instantly. Without one, it is an arithmetic question about sums — and that is where the frontier is.

Primary key $\rightarrow \|S(a)\| \rightarrow$ `Inner Unique: true`. The optimiser and the prover are tracking the same fact for different reasons.